Search your folders by what is in them

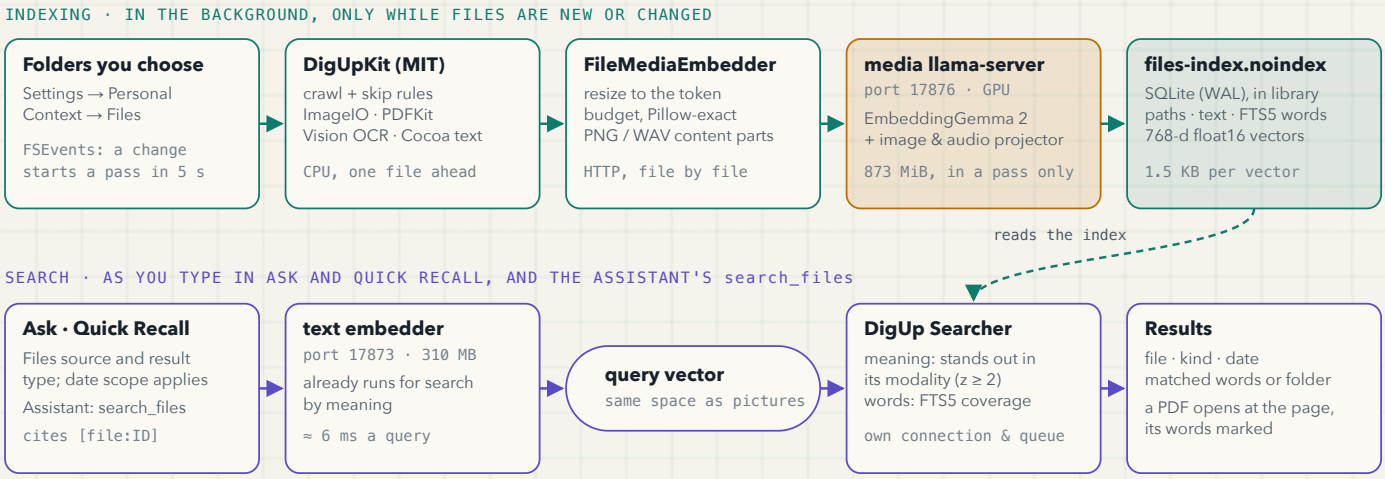
Files brings DigUp's file search into Ask. You choose folders; LokalBot reads the pictures, screenshots, PDFs and documents in them on this Mac, and Ask, Quick Recall and the Ask assistant can then find a file by what it says or shows, not only by its name.

"the clause about pets in the lease" → the PDF, opened at page 4

"payment declined error" → the screenshot

"a dog on the beach" → the photo

"INV-2041" → the invoice, by its exact words



One model, two servers.
Both run the same EmbeddingGemma 2 file LokalBot already uses for search by meaning (same revision, same SHA-256). Only the indexing server loads the 555 MB image and audio projector. Their text vectors are identical (cosine 1.000000), so a query from the small server scores pictures and PDF pages directly, and searching never loads the projector.

TAKEN FROM DIGUP

- The engine, `DigUpKit`, pinned by commit: crawler, skip rules, readers, index, search.
- Its image sizing, ported to Swift: matches Pillow bit for bit.
- Its ranking: meaning per modality plus exact words.

ADDED BY LOKALBOT

- An embedder that talks to LokalBot's own llama-server, not DigUp's llama.cpp build.
- Ask, Quick Recall, the assistant's tool and `[file:ID]` citations.
- Pauses for recordings, Low Power Mode and heat; Settings with a notice.

LEFT OUT FOR NOW

- DigUp's own panel, hotkey, updater and model downloader.
- Recordings and video (phase 3), and code search.
- The CLI and MCP: external agents can't read Files.

Try it. Settings → Personal Context → Files → *Search my files*, then *Add Folder…* for Documents, Downloads or a photo folder; Settings shows each pass as it runs. In Ask, type what you are looking for: files follow meetings and screen moments, and *Sources* → *Result type* → *Files* shows only files. Quick Recall lists them under Files. Ask a question, and the assistant searches them too and cites the file it used.

INDEXING

One indexing pass, step by step

A pass runs at launch, when the folder list changes, and five seconds after something in the folders changes. It reads only what is new or changed, and the image model loads only when there is something to read.

1. **Trigger** — `launch · folders · FSEvents + 5 s`
   Changes inside hidden folders are ignored. A burst of changes becomes one pass.

2. **Should it wait?** — `checked before each batch`
   A meeting is recording, Low Power Mode is on, or the Mac is hot (thermal state serious or critical). Then it waits, and starts again when that ends.

3. **Sync** — `CPU · no model`
   DigUp walks the folders and compares sizes and dates with the index: new and changed files are queued, moved files keep their vectors, deleted files and folders taken off the list leave the index.

   > **Nothing new?** The pass ends here. An unchanged library never loads the image model.

4. **Prepare the model** — `first time only`
   Downloads the 555 MB projector (and the 310 MB text model, if search by meaning never ran), each checked against its pinned size and SHA-256.

5. **Start the media server** — `≈ 1 s · 873 MiB`
   A llama-server on port 17876, held in memory for the pass.

6. **Read and embed** — `CPU reads · GPU embeds`
   Screenshots first, then pictures, PDFs and documents, newest first. While the GPU embeds one file, a CPU thread decodes, OCRs or extracts the next. Long PDFs get their first 30 pages now.

7. **Finish long files** — `30 pages · 40 chunks a step`
   The rest of PDFs over 50 pages and of long documents, once everything else is searchable.

8. **Let go** — `2 min · 10 min`
   The media server stops two minutes after the pass. Search vectors leave memory after ten minutes without a search.

WHAT ONE FILE BECOMES

KIND	READ WITH	VECTORS
Screenshot	ImageIO; Vision OCR for exact words	1 image · 280 tokens
Picture	ImageIO, EXIF date	1 image · 140 tokens
PDF page	PDFKit text, up to 4,000 characters	1 text per page
Scanned page	rendered page; OCR words	1 image · 280 tokens
Document	Cocoa text, ~1,800-character chunks	1 text per chunk
Recording, video	not read yet: excluded by extension until moments open in place	

NEVER READ

- Hidden files and folders, `~/Library`, app bundles, Photos libraries.
- Caches: `CACHEDIR.TAG`, `node_modules`, `DerivedData`, `.noindex`.
- Inside code projects, everything but screenshots.
- Folders that look like datasets (300+ machine-named media files).
- iCloud files that are only in the cloud: never downloaded.
- Pictures under 256 px, links, copy-protected audio.

Outages don't cost files. If the media server stops answering, DigUp ends the pass and keeps the files queued for the next one. Only a file that can't be read, like a broken picture, is marked failed, and it is read again when it changes.

COMPUTE ON YOUR MAC

How much your M4 Max works for it

Measured on this MacBook (M4 Max, 48 GB, on AC power) with the app's own llama-server and settings. Model time runs on the GPU; reading files runs on a CPU thread at the same time.

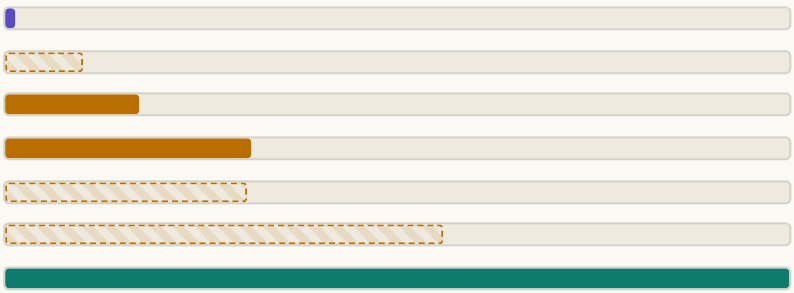
0 B
sent off this Mac by indexing or searching.

555 MB
one download, the first time: the image and audio projector.

873 MiB
while a pass runs (peak 1,002). Gone 2 minutes after it.

≈ 6 ms
to embed a search, on the text server that already runs.

MODEL TIME PER ITEM · P50



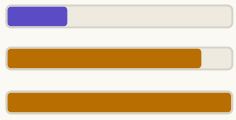
Striped: not indexed by Files yet, or for comparison. Sizing each image for its token budget is what took a screenshot from 574 tokens and 247 ms (13 October run) to 277 tokens and 139 ms. One image per request; batching eight saved nothing.

FIRST PASS · MODEL TIME

FOLDER HOLDS	GPU TIME
2,000 screenshots	≈ 4.6 min
10,000 photos	≈ 13 min
5,000 PDF pages of text	≈ 2–3 min
A day's 20 new screenshots	≈ 3 s
Nothing changed	0 s

From the per-item times above; PDF pages from text throughput (≈ 14 ms per 1,500 characters). Reading runs on the CPU at the same time. DigUp measures Vision OCR at about 0.24 s per Retina screenshot, so for screenshots, reading rather than the GPU sets the pace.

MEMORY · FOOTPRINT



The text server is the one search by meaning already keeps. The media server exists only during a pass and counts against LokalBot's model memory budget, so it can't push out a model you are using. Search vectors take 1.5 KB each, loaded on the first search and freed after ten idle minutes: 20,000 pages and pictures are about 30 MB.

When it runs. Only during a pass, and only for files that are new or changed. When a meeting starts recording, Low Power Mode comes on or the Mac gets hot, it stops after the files in hand and waits. Typing a search costs one 6 ms embedding on the server that is already running, plus a scan of the vectors in memory.

QUALITY AND BOUNDARIES

Why the results can be trusted

The vectors match DigUp's measured pipeline, the ranking is the one DigUp tested, and the feature has the same boundaries as LokalBot's other sources.

SAME VECTORS AS DIGUP · COSINE TO ITS REFERENCE

INPUT	TOKENS	COSINE
Texts (English, Bengali, Arabic, 1,800-token document)	–	**≥ 0.999999**
Screenshot (PNG), budget 280	277	**1.000000**
Speech (WAV)	104	**1.000000**
Photo (JPEG), budget 140	134	**0.99964**
Photo as video frame, budget 70	67	**0.99860**
Screenshot, llama.cpp's own sizing	574	0.98981
Photo, llama.cpp's own sizing	340	0.97075

Through the app's Swift path and a live media server (`FileMediaEmbedderReferenceTests`). JPEG inputs sit just under 1.0 only because macOS decodes JPEG with ImageIO and DigUp's reference with stb_image; fed the same pixels, the paths agree exactly.

RANKING

Each search scores every vector, then centres the scores within each modality, because text scores higher than pictures for the same topic. A file counts when it stands out by meaning ($z \geq 2$) or holds at least half the query's words; exact words add a bonus, so invoice numbers and names win. On DigUp's evals this found the right file first 88% of the time, against 73% with reciprocal-rank fusion, the method LokalBot uses for screens today (synthetic eval; 86% against 82% on real files).

TESTS IN THIS PULL REQUEST

- `FileSearchServiceTests`: DigUp's real crawler, readers and index.
- `FileMediaEmbedderTests`: request shapes, outages, broken files.
- `FileImageSizingTests`: DigUp's Pillow fixtures, bit for bit.
- `FileSearchAskTests`: source, tool, citations, evidence.
- `FileMediaEmbedderReferenceTests`: the live-server check above.

PRIVACY BOUNDARIES

- Off until you turn it on after a notice; nothing is read until you choose a folder.
- Reads only the listed folders; never changes, moves or uploads a file.
- Turning Files off deletes the whole index at once.
- A remote Think model sees file passages only when you ask with Files selected, and only at an origin you approved.
- External agents (CLI, MCP) can't read Files.

WHAT COMES NEXT

- **Screen images.** The same media server can embed your retained screenshots. On your library (13 October) image vectors found the right screen first for 71 of 138 queries, against 44 for today's screen search.
- **Recordings and video.** 30-second sound windows and a frame per shot, opened at the moment.
- **A helper process** for indexing, so a runaway PDF can be stopped without touching the app.

WHERE IT LIVES

```
LokalBot/Services/FileSearch/ FileSearchService · FileSearchWorkers · FileMediaEmbedder · FileMediaEmbedderRuntime
· FileImageSizing
LokalBot/Views/ FileSearchSettings · FileSearchResultRow (PDF opener) · AskView · QuickRecallView · ChatView
(citations)
LokalBot/Models/ AskRouting (source, result type, tool scope) · RecallWorkspaceState (search)
LokalBot/Services/Chat/ MeetingChatTools (search_files) · ChatCitation ([file:ID]) · ChatAgent (prompt)
LlamaServerRole.mediaEmbedder (17876) · project.yml: DigUp at dc2651a · PRIVACY.md “Files you choose”
```